BENCHMARK

TPC Benchmark DS 4.0.0

Decision-support data set and toolkit for the CDLE labs

Data set
24 tables, 3 sales channels, SF-scalable

Queries
99 templates in 4 classes

Metric
QphDS@SF from six timed tests

Toolkit
dsdgen + dsqgen, ready in an hour

CDLE • MEIC/MEIM • ISEL

Contents

Everything you need to understand the benchmark and run its data set in the lab

01 What TPC-DS is

A decision-support benchmark from the TPC
The retail business model: stores, catalogue, web
What a result measures

02 The data set

Snowflake schema: 7 fact tables, 17 dimensions
Six ER diagrams: overview, channels, inventory, dimensions
Scale factors: lab sizes and published SFs

03 Workload and metric

99 query templates in four classes
Data maintenance from source flat files
Six tests, one number: QphDS@SF

04 Hands-on with the toolkit

What is in the download
dsdgen: generate the data
dsqgen: generate the queries; load into Spark

TPC-DS measures a decision-support system end to end

A benchmark from the Transaction Processing Performance Council (TPC); version 4.0.0 dates from November 2024

What it models

- Large volumes of data and real-world business questions
- Queries of mixed complexity: ad hoc, reporting, iterative OLAP, data mining
- High CPU and I/O load, periodic refresh from source OLTP systems
- Runs on RDBMS and on Big Data engines such as Hadoop and Spark

What a result measures

- Query response time in single-user mode (Power Test)
- Query throughput in multi-user mode (Throughput Tests)
- Data maintenance performance (refresh runs)
- One metric: QphDS@SF, plus price-performance $/kQphDS@SF

Why it matters for CDLE

Realistic schema | **Free data generator** | **99 SQL queries** | **1 GB to 100 TB** | **Published results**

Source: TPC-DS Standard Specification 4.0.0, clauses 0.1 and 7.6.

A nationwide retailer selling through three channels

Any industry that manages, sells and distributes products; TPC-DS does not benchmark the operational systems, only the warehouse

Store
Brick-and-mortar shops nationwide

store_sales / store_returns

Ad hoc part of the schema

Catalogue
Mail-order catalogue pages and call centres

catalog_sales / catalog_returns

Reporting part: 40% of all data

Web
Web sites and web pages

web_sales / web_returns

Ad hoc part of the schema

Shared by all channels: a simple inventory system (warehouses), a promotion system, customer profiles (CRM) and common dimensions such as item, date and time

Business processes modelled

- Record customer purchases and returns from any channel
- Modify prices according to promotions
- Maintain warehouse inventory
- Create dynamic web pages
- Maintain customer profiles (CRM)

Design assumptions worth knowing

- The three channel sub-systems are autonomous: they may hold redundant customer and address data
- The warehouse tracks the operational systems with some delay, through data maintenance
- Queries can be long-running and multi-part; the system is quiescent for queries in a given period

Source: TPC-DS Standard Specification 4.0.0, clauses 1.2, 1.3 and 1.4.1.

A snowflake schema: 7 fact tables and 17 dimensions

Every dimension has a single-column surrogate key (…_sk); fact tables join to dimensions only through those keys

7 fact tables

store_sales · **store_returns**
catalog_sales · **catalog_returns**
web_sales · **web_returns**
inventory *(catalogue and web channels only)*

One row per sale line item or return line item; composite PK such as (ss_item_sk, ss_ticket_number)

17 dimension tables

- customer, customer_address, customer_demographics, household_demographics, income_band
- item, promotion, reason, ship_mode
- store, call_center, catalog_page, web_site, web_page, warehouse
- date_dim, time_dim

Plus dsdgen_version, an auxiliary table not used by the benchmark

Dimension types

Static loaded once, never changes (date_dim)

Historical keeps history with several rows per business key and validity dates; facts point to the row active at the time (item)

Non-historical updates overwrite, facts see the current value (customer)

Reading a column name

Prefix = table: ss_ store_sales, sr_ store_returns, cs_/cr_ catalogue, ws_/wr_ web, inv_ inventory, i_ item, c_ customer, d_ date_dim, t_ time_dim, s_ store, w_ warehouse, p_ promotion, cd_/hd_ demographics, ca_ address

Suffix _sk = surrogate key (PK or FK) · _id = business key · (B) in the spec marks business keys used only for data maintenance

Source: TPC-DS Standard Specification 4.0.0, clauses 1.3.3, 2.1 and 2.2.

ER overview: 26 foreign keys link the 11 core entities

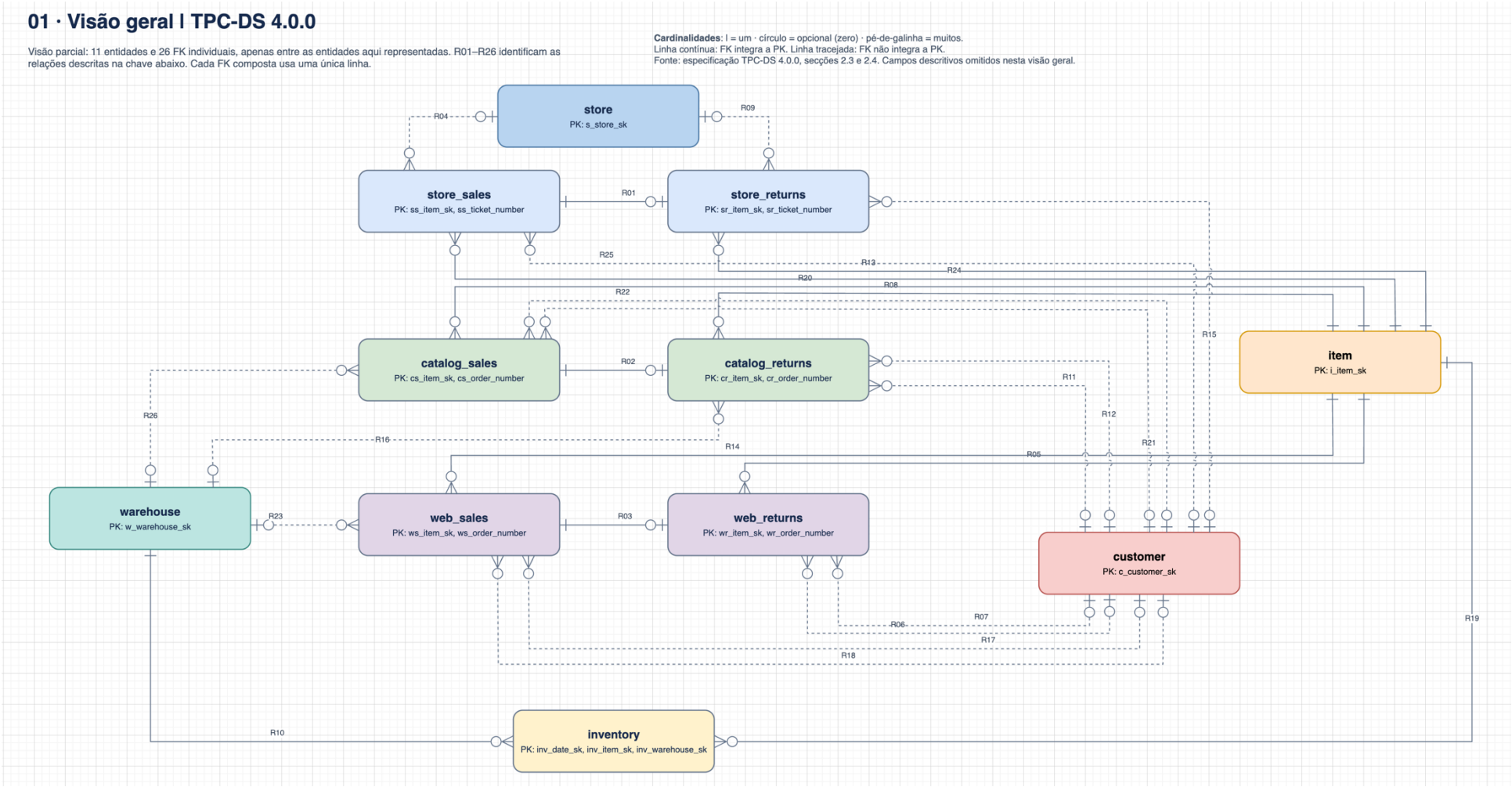


Diagram 01, built from clauses 2.3 and 2.4. Solid line: the FK is part of the PK; dashed: it is not. Circle = optional, crow's foot = many. Descriptive columns omitted.

Store channel: a sale line joins up to 9 dimensions

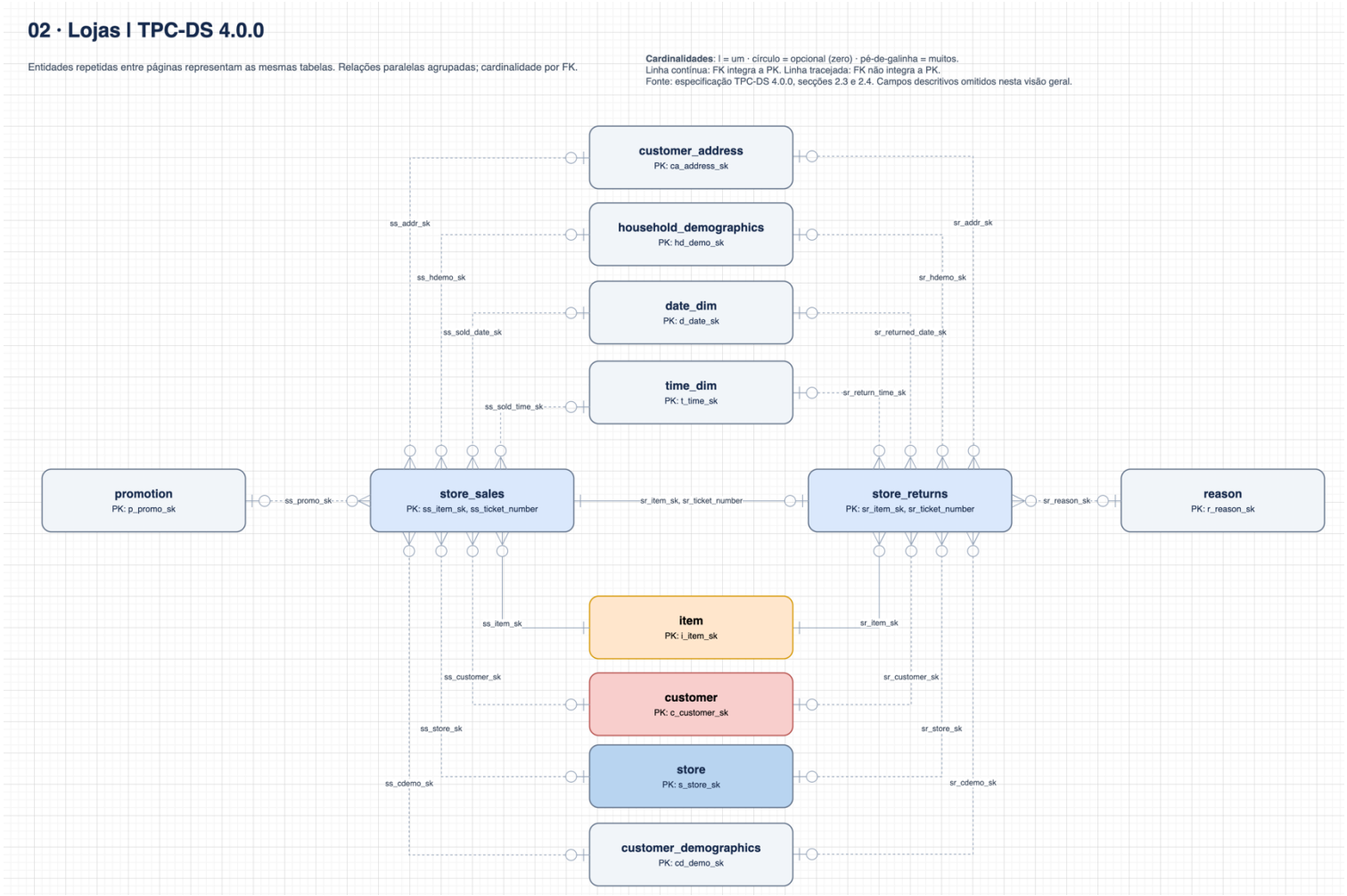


Diagram 02. store_sales and store_returns share the composite key (item_sk, ticket_number); a return may reference a reason. Same notation as the overview.

Catalogue channel: one dimension, two roles (bill/ship)

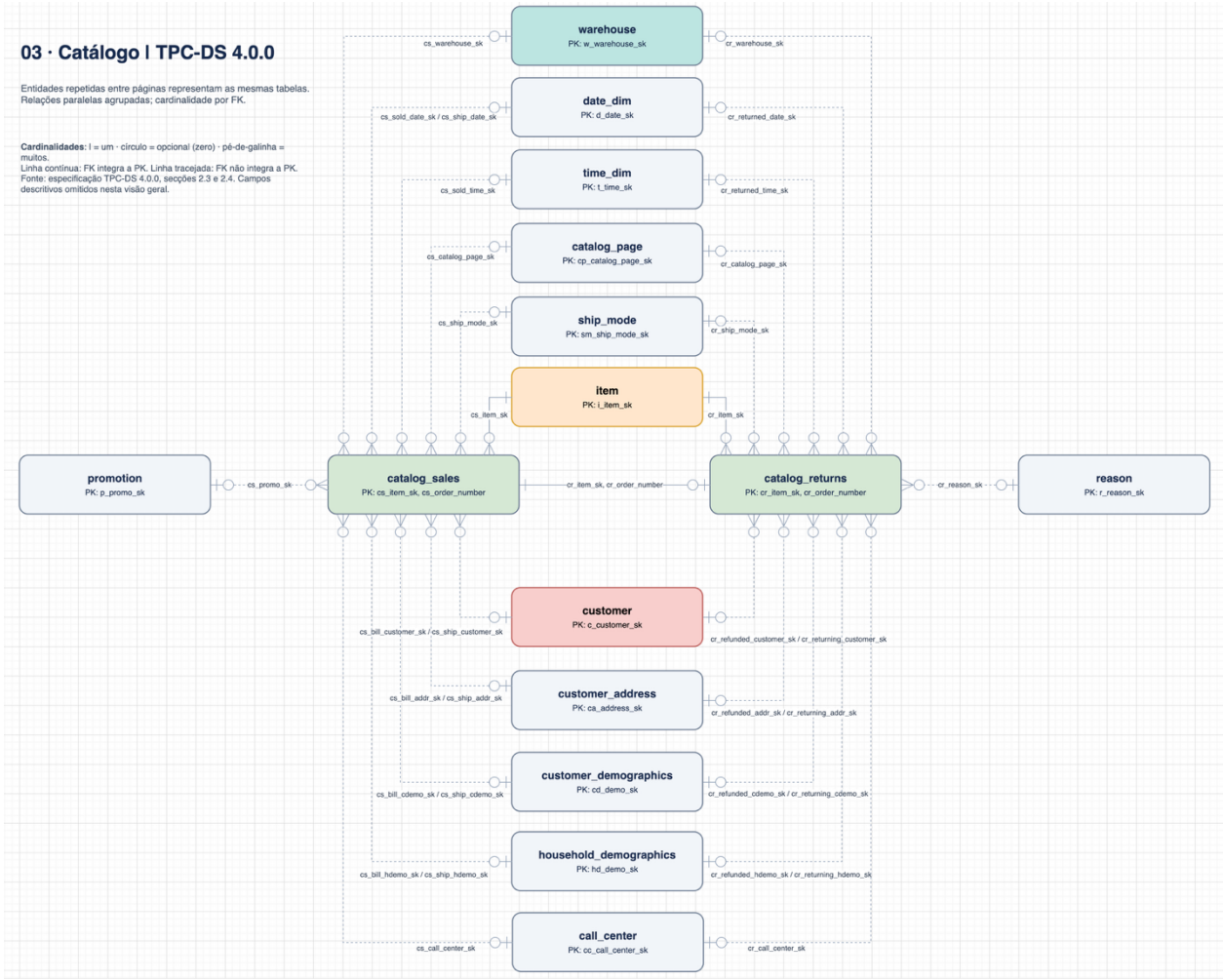


Diagram 03. The same dimension is referenced twice under different roles (cs_bill_* / cs_ship_*, cr_refunded_* / cr_returning_*); the catalogue channel also uses warehouse, ship_mode, catalog_page and call_center.

Web channel: the same pattern plus web_site and web_page

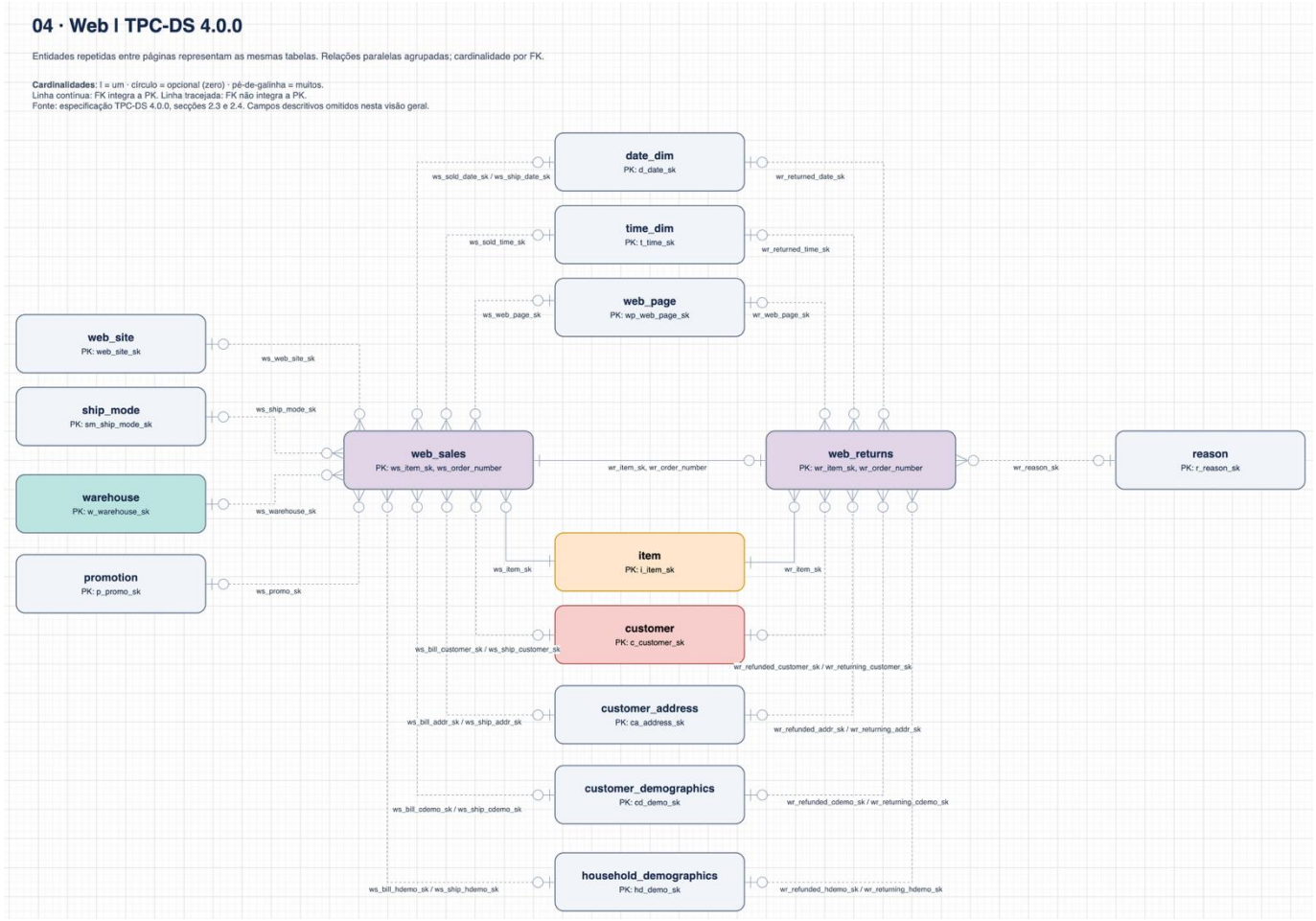


Diagram 04. web_sales and web_returns share the composite key (item, order number); like the catalogue, bill/ship and refunded/returning roles reference customer and its demographics twice. Web-specific dimensions: web_site and web_page.

Inventory: one row per item, warehouse and week

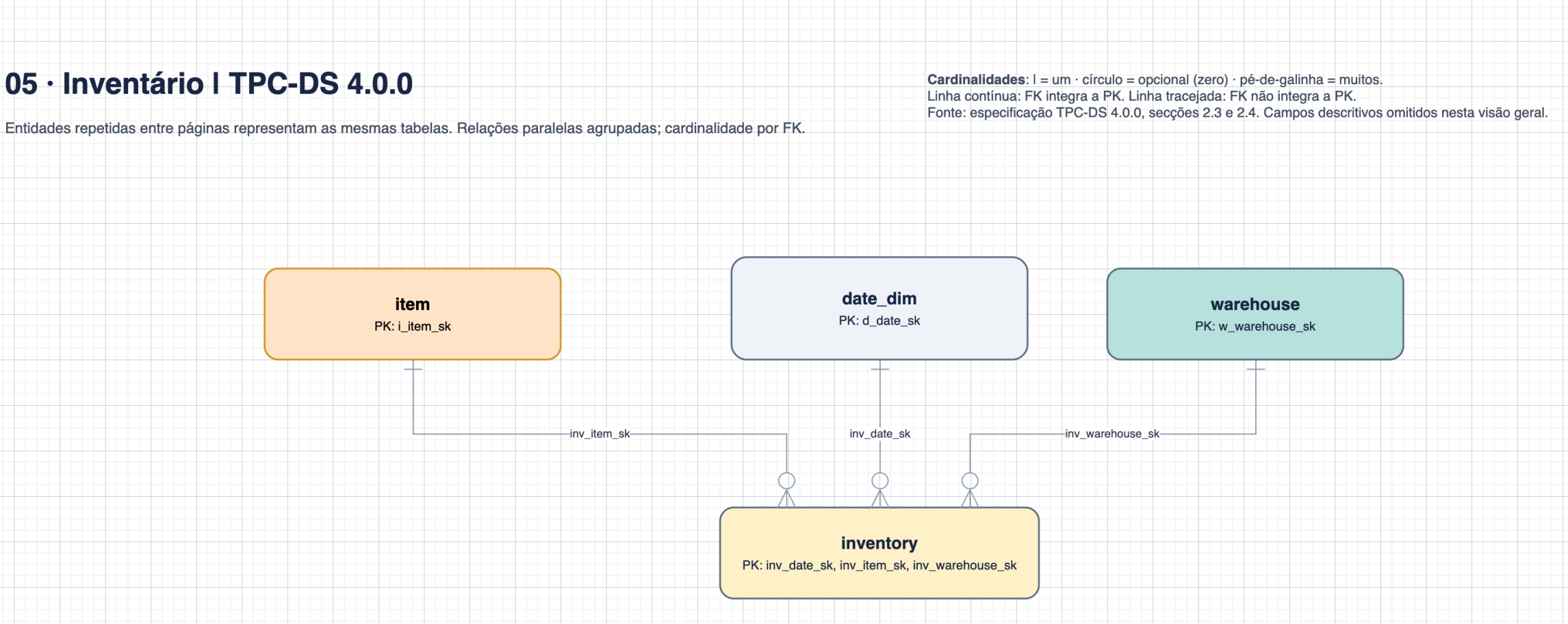


Diagram 05. inventory has a three-column composite PK made entirely of foreign keys; at 1 TB it holds 783 million rows, the largest non-sales table.

Dimensions: snowflake links between dimension tables

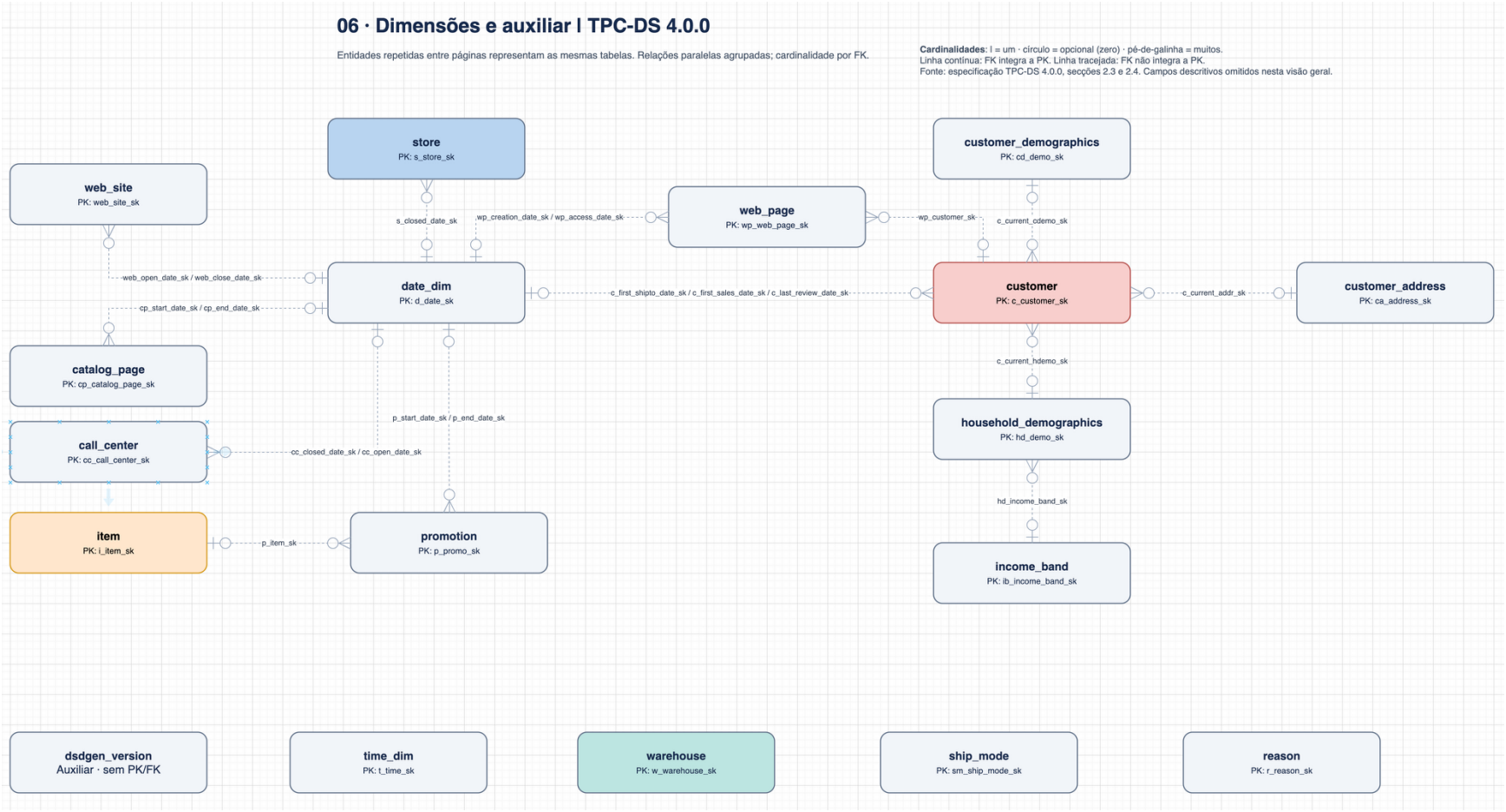


Diagram 06. Dimensions reference each other: customer points to address, customer and household demographics (then income_band); store, web_site, web_page, call_center, catalog_page and promotion carry date_dim keys; promotion also references item. The rest stand alone.

Lab-scale data sets: three fact tables are 85% of the bytes

Any SF works for teaching: the 1 GB, 10 GB and 100 GB sets were generated with dsdgen for the CDLE labs (1 GB is also the qualification size)

Table	1 GB rows	1 GB size	10 GB size	100 GB size	Share (100 GB)
store_sales	2,880,404	368 MB	3.7 GB	37.9 GB	39.7%
catalog_sales	1,441,548	281 MB	2.8 GB	28.8 GB	30.2%
web_sales	719,384	139 MB	1.4 GB	14.3 GB	15.0%
inventory	11,745,000	214 MB	2.4 GB	7.7 GB	8.0%
store_returns	287,514	31 MB	320 MB	3.2 GB	3.4%
catalog_returns	144,067	20 MB	210 MB	2.1 GB	2.2%
web_returns	71,763	9 MB	97 MB	998 MB	1.0%
customer	100,000	13 MB	63 MB	255 MB	0.3%
all 24 tables	**≈ 18.6 M**	**1.15 GB**	**11.2 GB**	**95.4 GB**	**100%**

How it was generated

dsdgen -SCALE 1 / 10 / 100 with -PARALLEL 16: one .dat chunk per child (store_sales_7_16.dat); pipe-delimited, empty field = NULL.

What grows, what does not

Sales, returns and inventory grow with SF; date_dim, time_dim, customer_demographics and the small dimensions do not.

Sizing for Spark

Parquet is **3.5× smaller** than text (DuckDB, Snappy): 95 GB → 27 GB at SF 100, 11.2 → 2.9 GB, 1.15 → 0.28 GB. 10 GB fits a laptop; 100 GB needs the cluster.

Sources: row counts from TPC-DS 4.0.0 Table 3-2; text sizes from the .dat files (tpcds-llR2.txt), Parquet sizes from DuckDB output (parque-llR.txt), 14 Sep 2026.

Published results use five scale factors: 1 TB to 100 TB

SF is roughly the raw size in GB; a 100 TB store_sales holds 288 billion rows, 100 000 times the qualification database

Table	1 TB (SF 1000)	3 TB (SF 3000)	10 TB (SF 10000)	30 TB (SF 30000)	100 TB (SF 100000)
store_sales	2,879,987,999	8,639,936,081	28,799,983,563	86,399,341,874	287,997,818,084
catalog_sales	1,439,980,416	4,320,078,880	14,399,964,710	43,200,404,822	143,999,334,399
web_sales	720,000,376	2,159,968,881	7,199,963,324	21,600,036,511	71,999,670,164
inventory	783,000,000	1,033,560,000	1,311,525,000	1,627,857,000	1,965,337,830
customer	12,000,000	30,000,000	65,000,000	80,000,000	100,000,000
item	300,000	360,000	402,000	462,000	502,000
date_dim	73,049	73,049	73,049	73,049	73,049

Only these five count

1, 3, 10, 30 and 100 TB are the permitted scale factors for a TPC-DS result; 100 TB is the current ceiling.

Not comparable across SF

Results at different scale factors must never be compared; the computational challenge changes with the volume.

Fact tables scale linearly

Sales rows grow with SF; customer and item grow slowly; date_dim (73 049 days) and time_dim (86 400 seconds) never change.

Source: TPC-DS Standard Specification 4.0.0, clause 3.1, Table 3-1 and Table 3-2 (selected tables).

99 query templates in four classes, one SQL per template

Templates are SQL:1999 with OLAP amendments plus substitution tags; dsqgen turns each into executable query text (EQT)

Reporting
Periodic, well-known questions; DBA can tune for them. Catalogue channel = reporting part (40% of data)

Ad hoc
Impromptu questions with little foreknowledge; store and web channels = ad hoc part

Iterative OLAP
Scenario-based sessions: a sequence of related queries exploring trends

Data mining
Large joins and aggregations returning big result sets for extraction

Example: query 3 (query3.tpl)

Business question: total extended sales price per brand of one manufacturer, for sales in a given month.

Substitution parameters: [MONTH], [MANUFACT], [AGGC] (qualification: 11, 128, ss_ext_sales_price)

```
select dt.d_year, item.i_brand_id brand_id, item.i_brand brand,
sum([AGGC]) sum_agg
from date_dim dt, store_sales, item
where dt.d_date_sk = store_sales.ss_sold_date_sk and
store_sales.ss_item_sk = item.i_item_sk and item.i_manufact_id =
[MANUFACT] and dt.d_moy = [MONTH]
group by dt.d_year, item.i_brand, item.i_brand_id
order by dt.d_year, sum_agg desc, brand_id [_LIMITC 100]
```

Rules that matter

- EQT must be used exactly as generated; only minor modifications (clause 4.2.3) or approved variants
- Parameters change with every stream: dsqgen -RNGSEED, -STREAMS
- Each stream is a distinct permutation of the 99 queries (Appendix D)
- Answer sets at 1 GB validate the output (clause 7.3)

Source: TPC-DS Standard Specification 4.0.0, clauses 1.4.1, 4.1 to 4.3 and Appendix B.3.

Data maintenance keeps the warehouse current

A warehouse is only as current as its sources: TPC-DS times the refresh too, unlike earlier decision-support benchmarks

1 Refresh set
One set of .dat files per refresh run (s_purchase, s_web_order, s_inventory…)

2 Source schema
Files map to the source tables of Appendix A; loading them is optional

3 DM functions
SQL or procedural SQL: all deletes first, then inserts and dimension updates

4 Visible to queries
Refresh runs never overlap; EADS (indexes, views) must be updated inside the run

Refresh data at SF = 1

- s_purchase_lineitem.dat: 12,264 rows, 1.3 MB (store sales)
- s_catalog_order_lineitem.dat: 6,138 rows (catalogue)
- s_web_order_lineitem.dat: 3,072 rows (web)
- s_inventory.dat: 540,000 rows, 27 MB (the largest file)
- Returns files for each channel, plus two small date-boundary files for the deletes

Rules in the benchmark

- One refresh run per query stream; each Data Maintenance Test runs Sq/2 of them
- Refresh sets are applied in the order dsdgen generated them
- PK/FK relationships must survive every run, enforced or not
- Historical dimensions (item) keep old rows; non-historical (customer) overwrite
- Staging areas allowed only during maintenance, and must be priced

Source: TPC-DS Standard Specification 4.0.0, clauses 1.5, 5.1 to 5.3 and Table 5-1.

Six timed tests, one number: QphDS@SF

The benchmark runs a fixed sequence: load, then single-user power, then two rounds of multi-user throughput and data maintenance

Load
Generate, load, build EADS, gather stats. Timed; enters the metric at 1%

Power
99 queries, one stream (query_0.sql), single session

Throughput 1
Sq streams in parallel, each a permutation of the 99

Data Maint. 1
Sq/2 refresh runs, in dsdgen order

Throughput 2
Sq more streams

Data Maint. 2
Sq/2 more refresh runs

$$QphDS@SF = \lfloor SF \times Q / \sqrt[4]{(TPT \times TTT \times TDM \times TLD)} \rfloor$$

Q = Sq × 99
total queries in one Throughput Test

TPT = TPower × Sq
power time scaled to the stream count

TTT = TTT1 + TTT2
elapsed time of both throughput tests

TDM = TDM1 + TDM2
elapsed time of both maintenance tests

TLD = 0.01 × Sq × TLoad
load time, weighted 1%

Also reported

Price/performance in $/kQphDS@SF, availability date and, optionally, energy in Watts/kQphDS@SF. All times in decimal hours, 1-second resolution.

Why it matters for the lab

You will time query_0.sql (power) and a few parallel streams on Spark: the geometric mean penalises any one slow phase, so a single pathological query hurts the whole result.

Source: TPC-DS Standard Specification 4.0.0, clauses 7.4.2 to 7.4.9 and 7.6. Results are only comparable at the same SF and major version.

PART 2 · Informations about the practice

what you will do in class labs

Using the TPC-DS toolkit: dsdgen, dsqgen and Spark

The toolkit: what is in the TPC-DS download

Free from tpc.org (TPC-DS_Tools_v4.0.0.zip, registration required); C sources, built with make on Linux or macOS; major.minor must match the spec

dsdgen tools/
Data generator: writes the 24 tables as pipe-delimited .dat files for any scale factor; also the refresh sets (-UPDATE)

dsqgen tools/
Query generator: turns the 99 templates into executable SQL for a chosen dialect, with random or qualification parameters

query_templates/ 99 templates
query1.tpl … query99.tpl plus the dialect files (ansi.tpl, netezza.tpl, oracle.tpl, sqlserver.tpl…) and templates.lst

query_variants/ Appendix C
Approved alternative phrasings (10a, 14a, 18a, 22a, 27a, 35a, 36a, 39a, 51a, 64a, 67a, 70a, 72a, 77a, 80a, 86a)

tpcds.sql · tpcds_ri.sql · tpcds_source.sql Appendix A
ANSI DDL for the warehouse, its referential-integrity constraints and the operational source schema

answer_sets/ Clause 7.3
Expected output of every query at 1 GB with the qualification parameters; two NULL orderings

data_maintenance/ Clause 5.3
Sample SQL for the refresh functions and the 5.3.11 views

specification.pdf · How_To_Guide Start here
This document and a non-binding walkthrough of the common tasks

Source: TPC-DS Standard Specification 4.0.0, Table 0-1 and Appendix F. The toolkit is versioned separately (third digit); use the latest 4.0.x.

dsdgen: generate the data in three commands

Run it once per scale factor; with -PARALLEL every child writes its own chunk, so the 16 children can run on 16 cores or 16 machines

```
# 1. build the tools (Linux/macOS; on macOS use OS=MACOS)
cd tools && make

# 2. generate 10 GB into ./sf10, 16 chunks, child 1..16
for c in $(seq 1 16); do
  ./dsdgen -SCALE 10 -DIR ../sf10 -PARALLEL 16 -CHILD $c -FORCE &
done; wait

# 3. check: 24 tables, ~11 GB of .dat files
ls -l ../sf10 | wc -l   &&   du -sh ../sf10
```

Flags you will use

`-SCALE n` SF in GB (1, 10, 100…)
`-DIR path` output directory
`-PARALLEL n -CHILD i` split into n chunks, write chunk i
`-TABLE name` one table only
`-UPDATE n` refresh set n for data maintenance
`-VALIDATE` reference data set for checks
`-FORCE` overwrite existing files

Output format

Text, one row per line, fields ended by '|' (trailing delimiter included). Two consecutive '|' = NULL; only NULL-able columns are ever empty.

File naming

table_child_total.dat, e.g. store_sales_7_16.dat. Small dimensions appear only in child 1; date_dim and time_dim are the same at every SF.

Watch out

Some string columns carry international characters (UTF-8). Row counts above 1 GB are approximate and dsdgen prevails over Table 3-2.

Source: TPC-DS Standard Specification 4.0.0, clauses 3.4 and 5.2; flag names from the toolkit's dsdgen -HELP. Check the exact syntax against your toolkit version.

dsqgen: turn 99 templates into runnable SQL

Templates carry substitution tags; dsqgen fills them from a seed and writes one file per stream, each a different permutation of the 99 queries

```
# 99 queries, 1 stream, ANSI dialect, SF 10 parameters
./dsqgen -DIRECTORY tpl -INPUT tpl/templates.lst \
   -DIALECT ansi -SCALE 10 -STREAMS 1 \
   -RNGSEED 20260914 -OUTPUT_DIR queries/sf10
# writes query_0.sql (power) and query_1.sql

# one template with qualification values (vs answer_sets/)
./dsqgen -TEMPLATE query3.tpl -DIRECTORY tpl -DIALECT ansi \
   -SCALE 1 -QUALIFY Y -OUTPUT_DIR queries/q3
```

Flags you will use

-DIRECTORY folder with the .tpl files
-INPUT list templates.lst: the 99 in order
-TEMPLATE t a single template instead
-DIALECT d ansi, netezza, oracle, sqlserver…
-SCALE n SF, drives parameter ranges
-STREAMS s writes query_0 … query_s.sql
-RNGSEED n reproducible parameters
-QUALIFY Y Appendix B qualification values

Dialects and Spark

There is no spark.tpl: start from ansi or netezza and fix the few non-portable bits (date arithmetic, LIMIT vs TOP, ROLLUP syntax).

Splitting the stream

query_0.sql is one file with 99 statements; split on the '-- end query N' markers into query01.sql … query99.sql for timing each one.

Validating

Run the qualification queries on the 1 GB set and diff against answer_sets/ (two NULL-ordering versions). Only then trust larger SFs.

Source: TPC-DS Standard Specification 4.0.0, clauses 4.1 to 4.3 and 7.3; flag names from the toolkit's dsqgen -HELP. Check the exact syntax against your toolkit version.

All three lab sets: generated and converted in 42 minutes

Wall-clock times for dsdgen with 16 children running at once, then a DuckDB COPY of the .dat files to Parquet; the 100 GB set alone takes 34 minutes

Scale	.dat size	Generate .dat (dsdgen)	Parquet size	Convert to Parquet (DuckDB)	Total
1 GB	1.15 GB	49 s	0.28 GB	3 s	52 s
10 GB	11.2 GB	6 min 51 s	2.9 GB	19 s	7 min 10 s
100 GB	95.4 GB	31 min 3 s	27.0 GB	2 min 44 s	33 min 47 s
All three	**108 GB**	**38 min 43 s**	**30.2 GB**	**3 min 6 s**	**41 min 49 s**

Generation is the cost

dsdgen writes 95 GB of text in 31 minutes; **converting it to Parquet takes under 3 minutes**. Conversion is 4 to 8% of the total at every SF, so writing Parquet once is cheap.

Not quite linear

1 → 10 GB takes 8.4× longer, 10 → 100 GB only 4.5×: the small dimensions cost the same at any SF. Conversion scales with size (3 → 19 → 164 s). Do not extrapolate 1 TB as 10 × 100 GB.

What it means for the lab

SF 1 is ready in a minute, SF 10 while you fetch a coffee; **generate SF 100 once** and share it on the cluster. Parquet is 3.5× smaller than the text at every scale; keep only the Parquet once the load is checked.

Measured 14 Sep 2026 on a MacBook Pro M3 (16 cores, 48 GB): dsdgen 4.0.0 -PARALLEL 16, all children concurrent; DuckDB 1.4.5, Snappy Parquet (TPCDS.txt, parque-llR.txt).

Load the Parquet into Spark and run the first query

The Parquet carries the tpcds.sql types (DECIMAL(7,2), DATE, not DOUBLE), so the load is a loop over 24 directories and a temp view each

```
# PySpark: register the 24 Parquet tables written by DuckDB
import os
base = "tpcds/sf10"
for t in os.listdir(base):
    spark.read.parquet(f"{base}/{t}") \
         .createOrReplaceTempView(t)

# expect 188 int, 148 string, 80 decimal, 12 date, 1 time columns
spark.table("store_sales").printSchema()
spark.sql(open("queries/sf10/split/query03.sql").read()).show()
```

Pitfalls when reading the .dat

In the DuckDB COPY, or in Spark's CSV reader

- Trailing '|' creates a 25th empty column: drop it or add a dummy field
- Empty fields must become NULL, not empty strings
- char(N) columns are space-padded; trim before comparing
- decimal(7,2) and identifier: DECIMAL and BIGINT, not double/int
- Dates in date_dim are 'YYYY-MM-DD' strings; cast to DATE

No DuckDB? Read the text

spark.read.csv with sep "|", emptyValue None and the tpcds.sql schema; **write Parquet once** before timing anything.

Partition the big three

store_sales, catalog_sales and web_sales by *_sold_date_sk (or by year via date_dim); keep dimensions unpartitioned and broadcast them.

What to measure

Wall-clock per query (query01 to query99), then 2 to 4 streams in parallel; compare 1 GB vs 10 GB vs 100 GB and text vs Parquet.

Schema and NULL rules: TPC-DS Specification 4.0.0, clauses 2.2 and 3.4.5, Appendix A. Parquet from DuckDB 1.4.5 (previous slide); Spark 3.x DataFrameReader options.